

Multi-Robot Task Auction

Statement ID: PNR2

Theme: Robotics, Drones and Autonomous Systems

Problem Statement:

Design a decentralized, auction-based task-allocation protocol for multi-robot fleets that dynamically assigns tasks based on real-time robot state and remains fully operational during severe communication failures.

Description:

Centralized task allocation in robotic fleets creates single points of failure and bottlenecks, especially in dynamic or hazardous environments where network connectivity is unreliable. This project requires building a decentralized protocol where autonomous robots actively bid on available tasks. The bidding engine must calculate optimal assignments by weighing multiple dynamic variables, including specific hardware capabilities, remaining battery life, physical proximity, operational uncertainty, and predicted completion time. Operating under strict production constraints, the architecture must ensure continuous operation through adversarial edge cases—such as sudden communication dropouts, network partitions, and conflicting bids—preventing task starvation and ensuring the fleet reliably completes its objectives even in heavily degraded states.

Features & Requirements

- **Multi-Component Architecture:** Decentralized communication nodes and localized bidding engines distributed across the robot fleet to eliminate single points of failure.
- **Dynamic Auction Mechanism:** Multi-variable bidding algorithms that continuously evaluate task suitability based on real-time battery, location, hardware capability, and environmental uncertainty.
- **Communication-Loss Resilience:** Robust peer-to-peer consensus and asynchronous allocation protocols that allow robots to intelligently claim and execute tasks even during severe network partitions.
- **Adversarial Edge Case Handling:** Built-in fault tolerance to resolve conflicting bids, handle Byzantine hardware failures, and recover dropped tasks without deadlocking the fleet.
- **Production-Ready Evaluation:** Measurable testing frameworks to quantify task completion rates, fleet resource efficiency, and protocol resolution latency under simulated and physical network degradation.

Level 1: Idea, Architecture, and Planning Establish the project's foundation by defining the problem and designing the technical blueprint. Key outputs include the Software Requirements Specification

(SRS), system architecture and data flow diagrams, UI wireframes, and a finalized tech stack and roadmap.

Level 2: Core Functionality Focus entirely on executing the Minimum Viable Product (MVP). This phase involves setting up the database schemas, writing the primary business logic, developing essential APIs, and delivering a functional application where users can complete core tasks.

Level 3: Intelligence, Security, and Real Deployment Transform the local MVP into a secure, public-facing application. This involves integrating advanced algorithms or AI features, implementing strict security measures (authentication, encryption), configuring automated CI/CD pipelines, and launching on cloud infrastructure.

Level 4: Scalability, Reliability, and Enterprise Readiness Prepare the live system to handle high traffic and ensure continuous operation. This requires containerizing the application for auto-scaling (e.g., Docker, Kubernetes), optimizing query performance with caching, and establishing active monitoring and alert dashboards.